
setter

A method that assigns data into an object instantiated from a class.

Getter and setter methods are designed to allow writers of client software to get data from and set data in an object, without needing explicit knowledge of the implementation of a class—specifically, without having to know or use the name of any instance variable. The `Person` class code in [Listing 8-1](#) has an instance variable `self.salary`. In [Listing 8-4](#) we add a getter and a setter to the `Person` class to allow the caller to get and set the salary, without providing direct access to the `Person`'s `self.salary` instance variable.

File: `PersonGettersSettersAndDirectAccess/Person.py`

```
class Person():
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    # Allow the caller to retrieve the salary
    ❶ def getSalary(self):
        return self.salary

    # Allow the caller to set a new salary
    ❷ def setSalary(self, salary):
        self.salary = salary
```

Listing 8-4: An example of a `Person` class with a getter and a setter

The `get` ❶ and `set` ❷ portions of these method names are not required but are used by convention. You generally follow these words with a description of the data being accessed, in this case `salary`. While it is typical to use the name of the instance variable being accessed, this is also not a requirement.

[Listing 8-5](#) shows some test code that instantiates two `Person` objects, then gets and sets their salaries using these getter and setter methods.